Language Detection Project

Pawan Bhattarai , Chuafa Vachoima, Thi Phuong Tran, Jens Naki, and Faisal Fahad Ibrahim Hawsawi

I. Problem Understanding

The task in this assignment is to learn a model binary classification and make it capable of distinguishing between human-written texts (label 0) and AI-generated texts (label 1). For this task the required raw text and preprocessing and feature engineering steps were already dealt with and the data set is already vectorised into embeddings generated by a pre-trained language model (roberta-base-openai-detector).

Therefore, our focus is on designing, optimising, and evaluating a deep learning model that can capture meaningful patterns in these high dimensional embeddings and generalise well to unseen data, while aiming to hit a score above the benchmark on the Kaggle public leaderboard, which is 0.89102.

From a machine learning perspective, this problem presents several challenges that we must address and will be expanded on later in this report. First, training data is provided at the sentence level, while validation and test data are labelled at the paragraph level, this requires a disassembly and an aggregation strategy to combine sentence-level predictions into a single label for a paragraph.

Early experiments show strong dependence on parameters such as learning rate, filter sizes, hidden layer sizes, scaling, and regularisation strength; small changes in these values can lead to large differences in performance, making systematic tuning crucial.

The validation set is relatively small (220 samples), which raises the concern that the evaluation is unstable and prone to variance from just a few misclassifications, this makes it difficult to rely on validation metrics for robust model selection.

Training data and validation/test data are generated from different datasets, which might make a gap in patterns learned during training and may not fully transfer to the validation/test data, reducing generalisation.

Lastly, the model relies on pre-generated embeddings from a large LLM, this makes the system less interpretable, our team cannot directly explain which textual features drive the model's decisions, limiting transparency, also models trained on older embeddings may fail to detect newer, more advanced outputs, highlighting the risk of distribution shift when the model encounters data from unseen LLMs.a

II. Data Exploration

Data inspection

We have one training set with each sample representing a sentence longer than 5 words and is word-embedded into a vectorized (100,768) tensor as features. Total samples of each training set are 8161 samples so we have a balanced data. Training sets are stored in numpy files with *.npy format.

The validation and test set are in slightly different format where the sets contain IDs and each ID represents a paragraph. There are different number of sentences in each ID. Only in the validation set, each ID is encoded as "0" or "1" to represent human-written paragraph and AI-generated paragraph respectively. **Figures 1 and 2** below explain the number of sentences in a single paragraph in the validation and testing set.

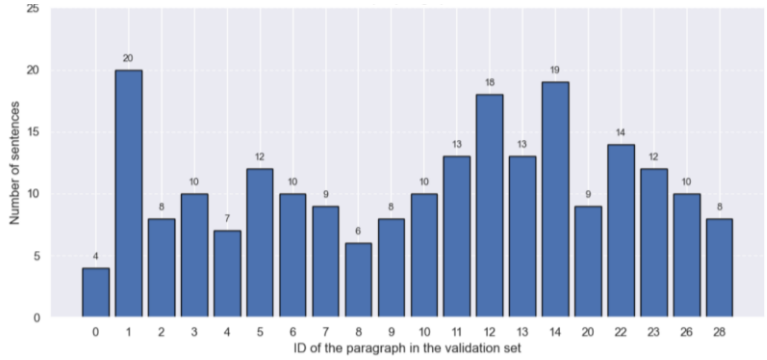

Fig. 1: Number of sentences per paragraph in the validation set

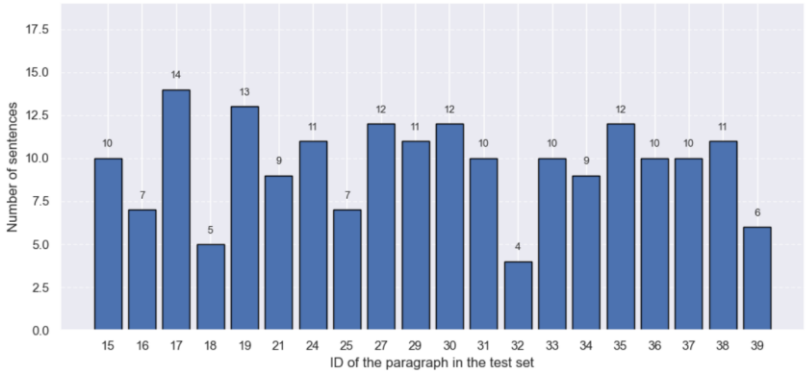

Fig. 2: Number of sentences per paragraph in the test set, for first 20 paragraphs only

Data pre-processing

After loading training, validation, and test set, we did data cleaning. For the training set, we combine AI-generated text training set and human-written text training set to be X_train. We created y_train where each sample is encoded as "0" or "1" to represent human-written texts and AI-generated texts respectively. We then proceed to horizontally concatenate y_train to X_train and reshuffle the data.

For validation and test set, Since each sample in the training is a sentence, we decided to extract sentences from each ID in the validation and testing set and assign the same ID to those sentences. Note that those sentences in each ID have the same shape as the sentences in our training set. Also, Since the training and validation sets are generated from different datasets, we can ensure there is no overlap between them, preventing data leakage.

The number of sentences sampled per class for training and validation set is illustrated by **Figure A1 (Appendix)**. Class distribution is not perfectly balanced in the validation set, as 130 sentences are human written and only 90 samples are AI-generated.

Data visualisation

Initially, we plot the distribution of word counts in the training set for each class, as explained by the **Figure** below. It appears that texts generated by AI generally use more words.

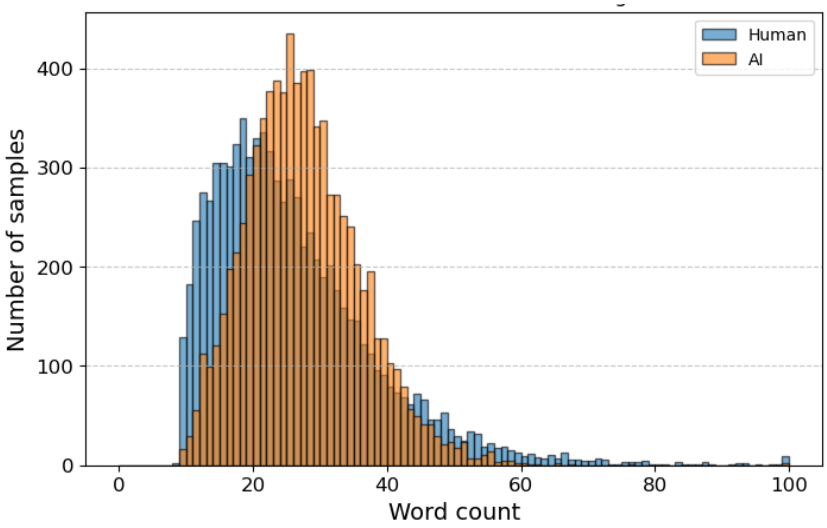

Fig. 3: The distribution of word counts in the training set for each class.

We also create heatmaps from 5 human-written and 5 AI-generated text samples to determine if there are any pattern differences between the two. Each heatmap illustrated by **Figure A2 (Appendix)** represents the first 15 words and their first 15 dimensions of embedding values.

In a glance, it is very difficult to come up with a distinct pattern between these two heatmaps. That is why we attempted to visualise the embeddings with two other ways, which are t-SNE and PCA visualisation .

Lastly, we generate t-SNE and PCA visualisation of the embeddings. These visualisations try to project the high dimensional embeddings into 2D space, so that we can inspect the overall structure and different clustering between human and AI generated text.

Figure 4 is the t-SNE plot of the training set embeddings. In the plot, class "1" represents AI text, denoted by yellow points. While class "0" is human text. denoted by purple points. There are clusters made up from sentences with similar characteristics. In most clusters, the composition is mixed between two classes, however some clusters are heavily dominated by one class. Clusters in the right-middle region for example are dominated by class "1".

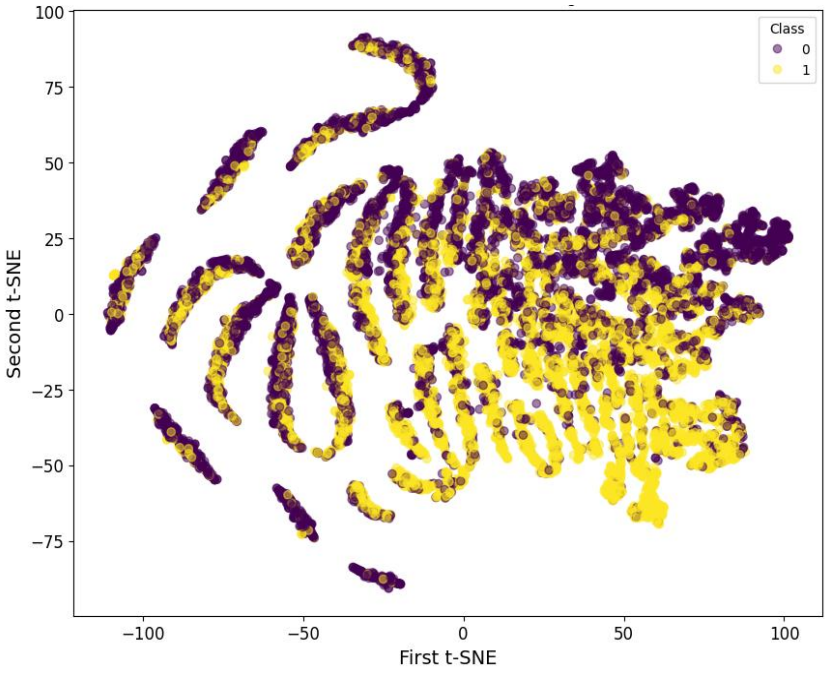

Fig.4 : t-SNE visualisation of the embeddings

The PCA plot projects the high dimensional embeddings to 2 dimension space of most important principal components or variance, as shown in **Figure 5**. Most points in the plot are overlapping heavily in the left-middle region, suggesting that class difference is not the component with biggest variance. Despite that, we can see that in some areas with less concentration of points, it is dominated by one class. For example, text with PC1 > 8 are mostly human written, while text with PC2 < -5 are mostly AI generated.

We can conclude from t-SNE and PCA plots that class separation is possible, but not simple. t-SNE and PCA model alone cannot be used to discern human and AI text since the classes are overlapping in most areas of both plots.

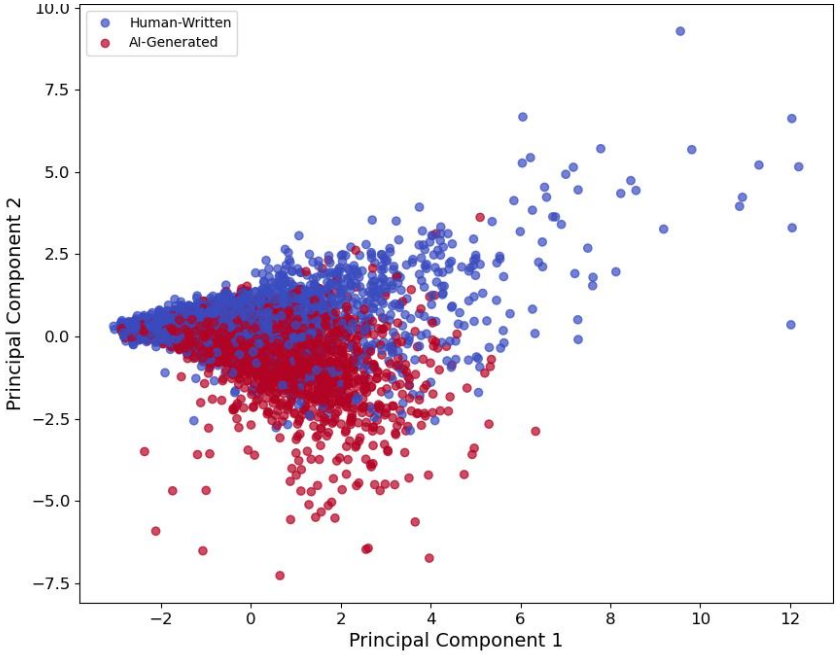

Fig. 5 : PCA of the embeddings using two Principal Components

Proposed workflow

We will train a model to predict each sentence on the test set using validation and training sets, as explained in **Figure 6** below. The predictions are then averaged based on their ID to obtain the probability for each paragraph.

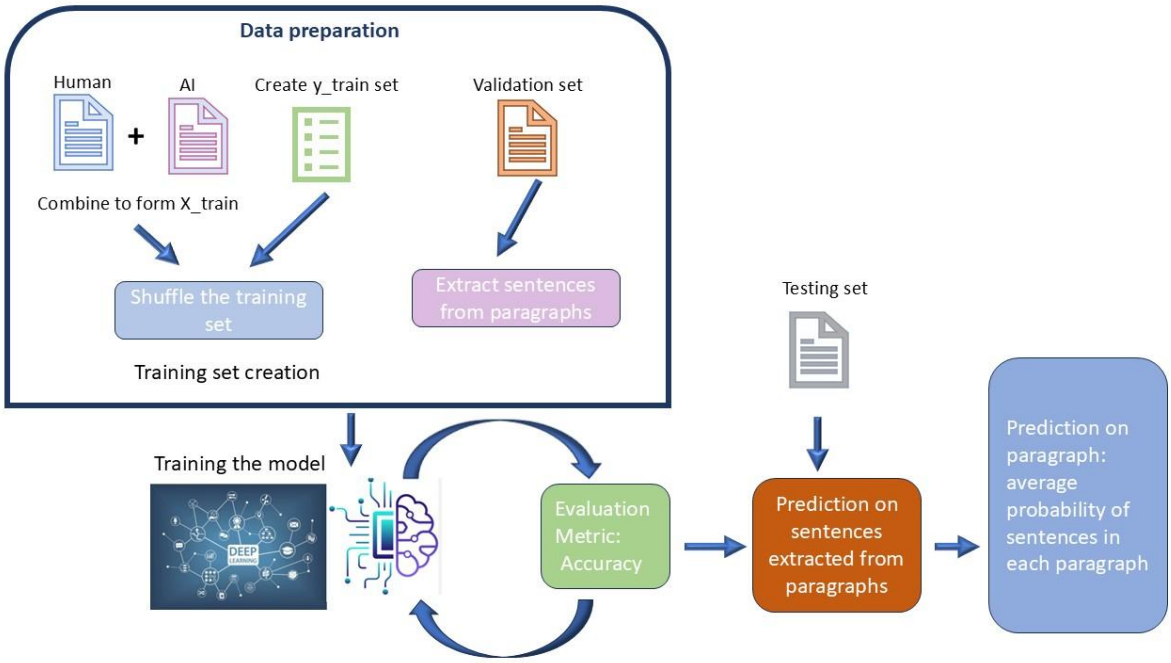

Fig. 6: Proposed workflow of the assignment

III. Modelling

Since the training data provided was generated using the RoBERTa-base model, we focused on finding AI-text-detection papers which use the BERT base model for the embedding step, and CNN for the classification. We found three papers with those requirements as explained in **Table 1** below.

Utku, A(2025) proposed a new model MILA, a framework for AI-text classification, combining BERT, CNN, and BiLSTM. The author also compared MILA with the convolutional 1D layer CNN as one of the baseline models. The others apply the TextCNN model, which is also a convolutional 1D layer CNN proposed by Kim, Y(2014). Thus, based on these researchers, we decided to use Conv1D CNN for classification of the dataset in this assignment.

Table 1: Literature review of papers with BERT base embeddings and CNN classification

No	Authors	Dataset	Tokenisation/ Embeddings and Classifier Models	Accuracy
1	Utku, A (2025).	ChatGPT classification dataset, publicly available: https://www.kaggle.com/datasets/mahdimaktabdar/chatgpt-classification-dataset/data 1018 article-level of lines, 50:50 written by human and AI, can be divided to 7344 sentence-level of lines, 3336 written by human and 4008 AI-generated	TF-IDF for Traditional ML approaches (SVM, XGBoost), BERT for deep learning models (CNN Conv1D, LSTM, BiLSTM, CNN-BiLSTM)	Sentence level accuracy: CNN-BiLSTM): 94.34% BiLSTM: 88.92% LSTM: 86.92% CNN: 85.70% XGBoost: 81.00% SVM: 77.60%(acc)
2	Eang, C & Lee, S (2024)	Stanford Sentiment Treebank v2 (SST-2)	BERT used for RNN and kNN, while RoBERTa used for TextCNN (Conv 1D)	RNN+BERT: 94.80% TextCNN+RoBERTa: 94.00% KNN+BERT: 92.31%
3	Liu et al. (2024).	ChatGPT Comparison Corpus dataset (HC3), from Guo et al (2023). ShareGPT GPT-4 Dataset, https://huggingface.co/datasets/shibing624/sharegpt_gpt4	BERT and TextCNN	Sharegpt dataset: 84.90% HC3 dataset: 81.90%

Our final model design has the architecture according to the following **Figure 7**. In part IV, we will further explain how we have arrived at this model architecture. We added three convolutional 1D layers which deems reasonable as the starting point. The first layer will learn low level local features. The second layer will combine lower level features and extract higher level features. The third layer will further capture more hierarchical complex features. We also added the max pooling layer to reduce the sequence length by only retaining important features, and thus, it helps to reduce computational time. Our final layer is the fully connected dense output layer for binary classification.

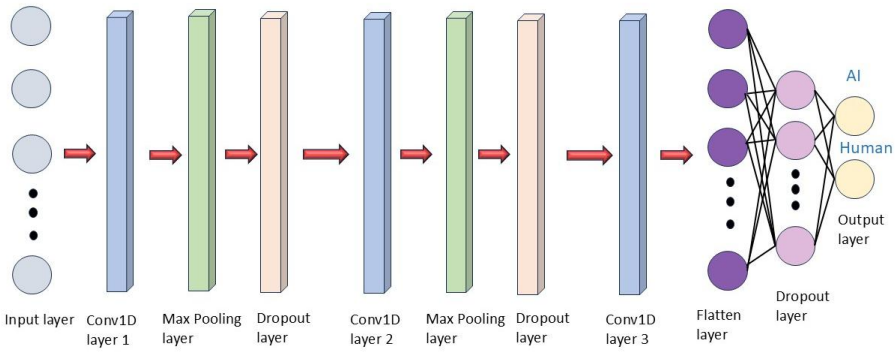

Fig. 7: Model final architecture

For optimization techniques, we have some fixed hyperparameters in our model and some other hyperparameters will be tuned. First, we set the kernel size to 3 which is common in text classification models as it can capture fine details. We also set the stride to 1 to ensure that the model does not skip important features. For the padding hyperparameter, we keep it as "Same" as we do not want any additional information in our model. We chose ReLu as our activation function as it is computationally fast and it reduces vanishing gradient issues. We chose Adam as our optimizer as it is known to be an effective optimizer and we want our learning rate to be able to adjust based on adaptive processes. We use accuracy as our evaluation metrics as we have balanced classes of human written-texts samples and AI-generated-texts samples in our training set. For the tuning of hyperameters, we have the number of filters in the convolutional 1D layers, the learning rate as base step size for Adam optimizer, the number of epochs and batch size. We utilize "call back" to restore the weights of the best epoch to obtain the best model. To speed up the training process, we also implement early stopping to stop the training if val_accuracy does not improve for 5 consecutive epochs.

IV. Result

We start with a simple architecture of CNN **(Figure A3, Appendix)** that has the following parameters: hiddensizes = [16,32,16], actfn = "relu", optimizer = keras.optimizers.Adam, learningrate = 0.001, batch_size = 8, and n_epochs = 15. The output of the training process is shown below:

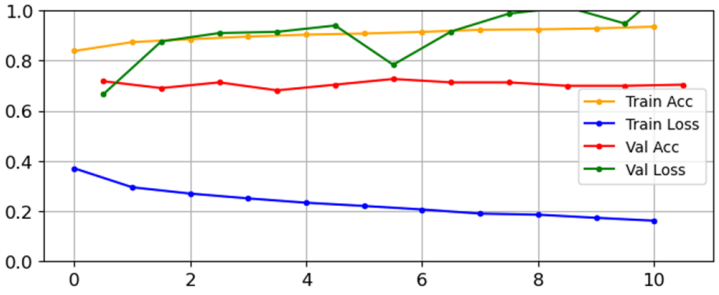

Fig. 8: Training process A

The training performance metrics show that the model fits well on the training data set. However, the validation loss varies and increases from 0.662 to 1.1430 after 11 epochs while the Train Loss decreases over the epochs. This indicates the possibility of overfitting, and the model does not generalise well on the unseen data. The variation in Val Loss may be explained by the small size of the unbalanced validation set (only 220 samples), which causes the model's performance metrics to be sensitive to incorrect classification. Additionally, since the validation set is generated from a different dataset than the one used for the training set, it may not be well representative.

To reduce overfitting, we use regularization by adding Dropout layers in the CNN and increase the batch size to 32 to improve generalisation. The result of the training process is as follows:

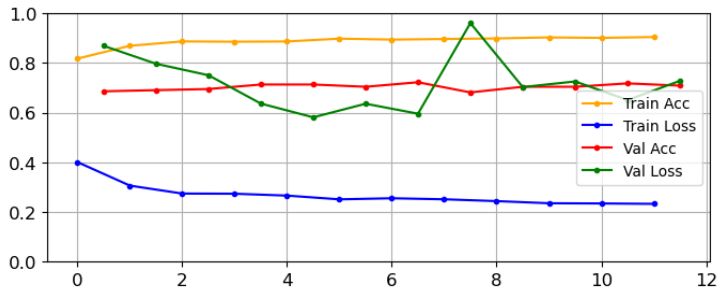

Fig. 9: Training process B with added drop-out layers and increased batch size

Figure 9 indicates that the model is still learning effectively with the updated architecture and the batch size of 32. The dropout layers help the Val Loss not decrease sharply compared to the previous model, resulting in reducing overfitting. The Val Loss drops to 0.5 which is lower than the previous training process's output. The model is improved and the predictions on the test set is at 0.93098 on Kaggle, higher than the current high benchmark (0.89120). However, we continue experimenting with different parameter values to enhance the model.

We lower the learning rate to 0.0001 to help improve generalisation and extend the number of epochs to 30. So we try with higher levels of "hiddensize" for deeper architectures, allowing it to capture more complex patterns. We also increase the batch size to 128 to observe if the model enhances its generalisation on the test set. The detail of parameters for each training process is described by **Table 2** below:

Table 2: Parameters for each training process

Training Process C	Training Process D	Training Process E
hiddensizes = [64, 128, 64] actfn = "relu" optimizer = keras.optimizers.Adam learningrate = 0.0001 batch_size = 32 n_epochs = 30	hiddensizes = [128, 256, 128] actfn = "relu" optimizer = keras.optimizers.Adam learningrate = 0.0001 batch_size = 32 n_epochs = 30	hiddensizes = [128, 256, 128] actfn = "relu" optimizer = keras.optimizers.Adam learningrate = 0.0001 batch_size = 128 n_epochs = 30

The plots of the training processes are as follows:

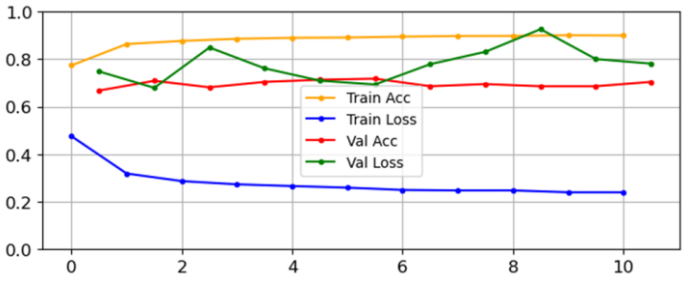

Fig. 10a: Training process C

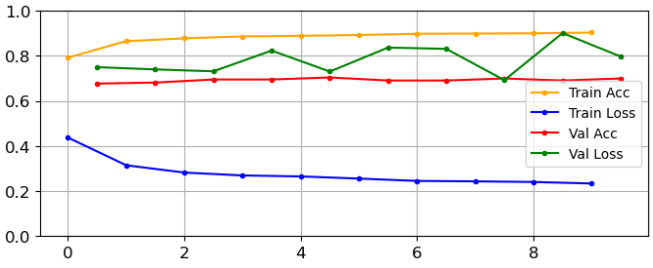

Fig. 10b: Training process D

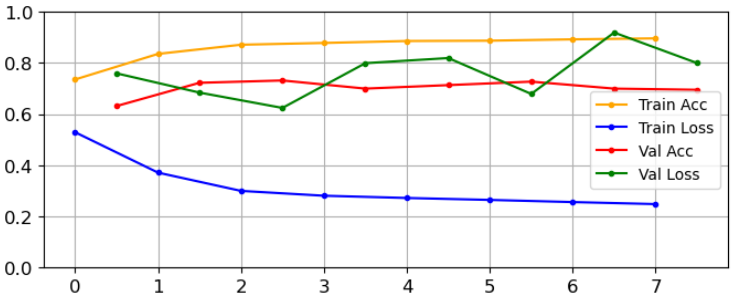

Fig. 10c: Training process E

The three plots show that all the training processes learn from the training data effectively. However, the model generated from the training process E performs the best on the validation set, producing the highest validation accuracy of 0.7318 and the lowest validation loss of 0.6844 among the best - performing models. Moreover, when generalising on the test set, the model from the training process E achieves the highest score of 0.9538, significantly surpassing the scores from the training process C (0.93772) and D (0.93149), indicating advanced generalisation. Hence, we choose the best model from the training process E as our final model.

V. Reflection

Our model was designed to classify individual sentences rather than the whole paragraph itself. In order to predict the paragraph, we have averaged the probability of prediction of each sentence within the paragraph. This method has smoothened the overall effect of prediction on the paragraph which might lead to inaccuracy. Furthermore, this method of training on sentences only enables the model to capture local patterns within the sentence but not as the whole paragraph, resulting in losing information between sentences within a paragraph.

We added dropout layers in our model to reduce overfitting. Although, overall, overfitting has reduced, it does not translate to no overfitting. It is still possible to overfit. Ideally, Train loss and Val loss should decrease simultaneously. Hence, the best generalization to the test data may not be achieved. We also used accuracy as our evaluation metric. However, on Kaggle, the evaluation metric used is AUROC. This might be the reason if our model does not generalize well to the remaining data of the test set in the private leaderboard.

Furthermore, as we have mentioned before, the test set and training set are generated from different datasets, so they might have different distributions. Therefore, the pattern learnt on the training set will not fit the test set well when distribution shifts. The result produced on the public leaderboard might just be a coincidence and does not represent the whole distribution of the test set.

References:

Eang, C & Lee, S 2024, 'Improving the Accuracy and Effectiveness of Text Classification Based on the Integration of the Bert Model and a Recurrent Neural Network (RNN_Bert_Based)', Applied Sciences, 8388, vol. 14, no. 18.S

Guo, B, Zhang, X, Wang, Z, Jiang, M, Nie, J, Ding, Y & Wu, Y 2023, 'How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection', arXiv preprint arXiv:2301.07597.

Kim, Y 2014, 'Convolutional Neural Networks for Sentence Classification', arXiv.Org.

Liu, Y, Li, M, Ye, Z, Shi, X, Wang, W & Zhao, H 2024, 'AI-Based Text Detection and Classification for Smart City Monitoring Using BERT and TextCNN Models', in 2024 International Symposium on Internet of Things and Smart Cities (ISITSC), IEEE, pp. 41–48.

Utku, A 2025, 'MILA: An Innovative Approach to Identifying AI-Generated Content Using BERT, CNN, and BiLSTM', Arabian Journal for Science and Engineering (2011).

Appendix

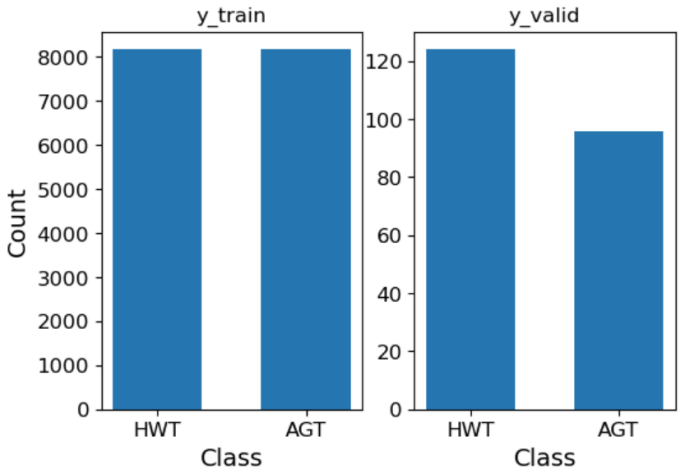

Fig. A1: The distribution of the human-written and AI-generated text samples in the training and validation

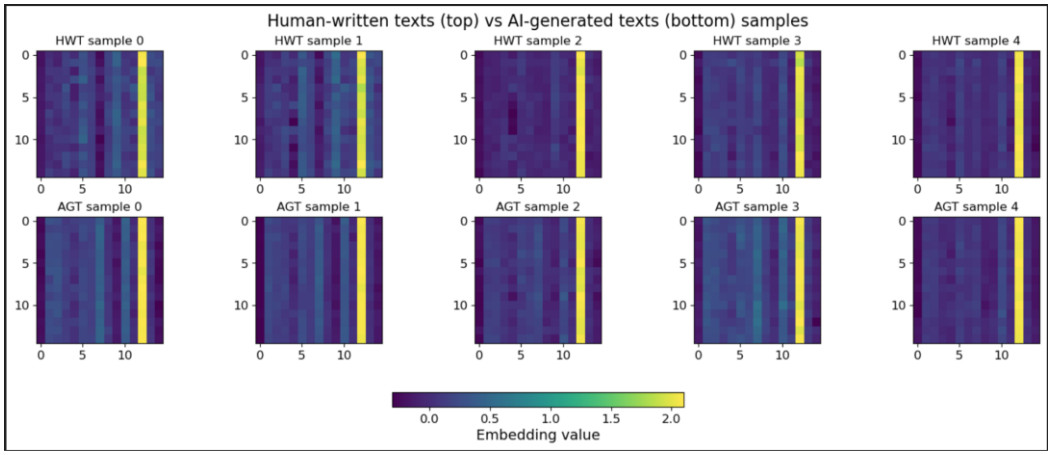

Fig. A2: heatmap of human-written and AI-generated texts

```python
def model_cnn_factory(hiddensizes, actfn, optimizer, learningrate):
    model = keras.models.Sequential()
    model.add(keras.layers.Conv1D(filters=hiddensizes[0], kernel_size=3, strides=1, activation=actfn, padding="same"))
    model.add(keras.layers.MaxPooling1D(pool_size=2))
    for n in hiddensizes[1:-1]:
        model.add(keras.layers.Conv1D(filters=n, kernel_size=3, strides=1, padding="same", activation=actfn))
        model.add(keras.layers.MaxPooling1D(pool_size=2))
    model.add(keras.layers.Conv1D(filters=hiddensizes[-1], kernel_size=3, strides=1, padding="same", activation=actfn))
    model.add(keras.layers.Flatten())
    model.add(keras.layers.Dense(1, activation = "sigmoid"))
    model.build(input_shape=(None, 100, 768))
    model.compile(loss="binary_crossentropy", optimizer=optimizer(learning_rate=learningrate), metrics=["accuracy"])
    return model
```

Fig. A3: Starting architecture of the CNN model